

Chapter 5. 토큰화 & Chapter 6. 임베딩(~TF-IDF)

Chapter 5. 토큰화

토큰화(Tokenization)

컴퓨터가 자연어를 이해할 수 있게 나누는 과정

- 컴퓨터가 텍스트를 보다 효율적으로 분석하고 처리할 수 있도록 하는 중요한 단계
- **토큰나이저(Tokenizer)** 사용: 텍스트 문자열을 토큰으로 나누는 알고리즘/소프트웨어
- 토큰나이저 구축 방법
 - 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
 - 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
 - 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
 - 머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다.
- OOV(Out of Vocab): 사전에 없는 단어나 토큰
- Curse of Dimensionality: 토큰이나 단어를 벡터화 하면 어휘 사전에 등장하는 토큰 개수만큼 차원이 필요하고, 벡터값이 거의 모두 0이 되는 희소 데이터로 표현
→ 희소 데이터 표현 방법은 어휘 사전에 등장하는 출현 빈도만 고려하기 때문에 문장에서 발생한 토큰들의 순서 관계 표현에 용이하지 않음

5.1 단어 및 글자 토큰화

5.1.1 단어 토큰화

단어 토큰화(Word Tokenization)

텍스트 데이터를 의미 있는 단어인 단어로 분리하는 작업

- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화 수행
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용

#예제 5.1 단어 토큰화

- split 메서드: 주어진 구분자를 통해 문자열을 리스트 데이터로 나누어줌
 - 구분자를 입력하지 않으면 공백(Whitespace)이 기준
- 출력결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

- '최고!', '최고!!', 'cg.'에서 알 수 있듯이 단어 토큰화는 한국어 접사, 문장 부호, 오타/띄어쓰기 오류 등에 취약

5.1.2 글자 토큰화

글자 토큰화(Character Tokenization)

띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식

- 장점
 - 비교적 작은 단어 사전 구축 가능
 - 작은 단어 사전을 사용하면 학습 시 컴퓨터 자원 절약 가능
 - 전체 말뭉치를 학습할 때 각 단어를 더 자주 학습 가능
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용

#예제 5.2 글자 토큰화

- 출력결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '불', '어', ' ', 'o', 's', 't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
```

- 글자 토큰화는 리스트 형태로 변환하면 쉽게 수행 가능
- 단어 토큰화와 다르게 공백도 토큰으로 나뉨
- 영어의 경우, 글자 토큰화 진행 시 각 알파벳으로 나뉨

- 한글의 경우, 하나의 글자는 여러 자음과 모음의 조합
 - 자소 단위 토큰화 수행 : 자모(jamo) 라이브러리 활용
- 자모 라이브러리: 한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리

```
#자모 변환 함수
retval = jamo.h2j(
    hangul_string
)
```

- 한글 문자열을 유니코드 U+1100~U+11FE 사이의 조합형 한글 자모로 변환하는 함수
- 한글 인코딩 방식은 조합형과 완성형으로 나눌 수 있음
 - 조합형: 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글 표현
 - 완성형: 조합된 글자 자체에 값을 부여해 인코딩하는 방식
- h2j함수: 완성형으로 입력된 한글을 조합형 한글로 변환
- j2hcj함수: 조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수

```
#한글 호환성 자모 변환 함수
retval = jamo.j2hcj(
    jamo
)
```

#예제 5.3 자수 단위 토큰화

- 출력 결과

[illegible]

- 단어 단위 토큰화에 비해 비교적 적은 크기의 단어 사전 구축 가능
- 접사와 문장 부호의 의미 학습 가능

- 작은 크기의 단어 사전으로도 OOV를 획기적으로 감소 가능
- 개별 토큰은 의미 X → 토큰의 의미를 조합해 결과 도출
 - 문장 생성, 개체명 인식 등 구현하는 경우, 다의어나 동음이의어가 많은 도메인에서 구별이 어려움
 - 모델 입력 시퀀스 길이가 길어질수록 연산량 증가

5.2 형태소 토큰화

형태소(Morpheme)

실제로 의미를 가지고 있는 최소의 단위

- 자립 형태소: 스스로 의미를 갖는 형태소
- 의존 형태소: 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 형태소

5.2.1 형태소 어휘 사전

어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전

- 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 등의 정보도 함께 제공
- 라이브러리 설치

```
!pip install konlpy
```

- 품사 태깅(POS Tagging): 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업 → 문맥 고려 가능

5.2.2 KoNLPy

한국어 자연어 처리를 위해 개발된 라이브러리로 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공

- 다양한 도구와 함수를 → 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에 사용
- 자바 언어 기반의 한국어 형태소 분석기로 자바 개발 키트 기반으로 개발됨
 - <https://www.oracle.com/java/technologies/downloads/>

- Okt, 꼬꼬마, 코모란, 한나눔, 메캅 등의 다양한 형태소 분석기를 지원

#예제5.4 Okt 토큰화

- Okt에서 지원하는 대표적인 메소드: 명사 추출(oka.nouns), 구문 추출(oka.phrases), 형태소 추출(oka.morphs), 품사 태깅(oka.pos)
- 명사 추출, 구문 추출, 형태소 추출 → 리스트 반환
- 품사 태깅은 각 단어의 품사 정보를 추출해 (형태소, 품사) 형태의 튜플로 구성된 리스트 반환

#예제 5.5 꼬꼬마 토큰화

- 명사, 문장, 형태소, 품사를 추출 가능
- 동일한 메서드라도 형태소 분석기 특징에 따라 다른 결과로 도출
⇒ 시스템의 목적과 환경에 맞는 적절한 형태소 분석기 사용이 필요

*Okt와 꼬꼬마에서 사용하는 품사 태그 정리

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EF0	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	궁정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존형 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

MAG	일반 부서	XPB	채언 접두사
MAC	접속 부서	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운데점, 콜론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

5.2.3 NLTK

자연어 처리를 위해 개발된 라이브러리로, 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공

- 주로 영어 자연어 처리를 위해 개발 but 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델 제공
- 라이브러리 설치

```
!pip install nltk
```

- NLTK 라이브러리를 활용해 토큰화나 품사 태깅 작업을 하기 위해서는 해당 작업을 수행할 수 있는 패키지나 모델을 다운해야함 → Punkt모델, Averaged Perceptron Tagger 모델 활용!
 - 두 모델은 트리뱅크라는 대규모 영어 말뭉치를 기반으로 학습됨
 - Punkt모델은 통계 기반 모델, Averaged Perceptron Tagger는 퍼셉트론을 기반 모델

#예제 5.6 패키지 및 모델 다운로드

```
import nltk

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")
```

#예제 5.7 영문 토큰화

- 단어 토큰라이저: 공백을 기준으로 단어 분리, 구두점 등을 처리해 각각의 단어를 추출해 리스트로 반환
- 문장 토큰라이저: 마침표(.), 느낌표(!), 물음표(?) 등의 구두점을 기준으로 문장을 분리해 리스트로 반환

#예제 5.8 영문 품사 태깅

- 출력결과

```
[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ','), ('can', 'MD'), ('create', 'VB'), ('the', 'DT'), ('impossible', 'JJ'), (',', ',')]
```

- 토큰화된 단어들이 들어가야 하며, 앞서 설치한 Averaged Perceptron Tagger 모델 설치 필요

*Averaged Perceptron Tagger모델에서 사용되는 품사 태그 정리

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

5.2.4 spaCy

사이썬(Cython) 기반으로 개발된 오픈 소스 라이브러리로서, NLTK 라이브러리와 마찬가지로 자연어 처리를 위한 기능을 제공

- NLTK 라이브러리와 주요한 차이점
 - 빠른 속도와 높은 정확도를 목표로 하는 ML기반
 - NLTK: 학습 목적, spaCy는 효율적인 처리 속도와 정확도를 제공하는 것을 목표
⇒ 더 크고 복잡하며 더 많은 리소스 요구
- 라이브러리 설치

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

- en_core_web_sm: 영어로 사전 학습된 모델
- GPU를 가속해 영어, 프랑스어, 한국어, 일본어 등을 비롯해 24개 이상의 언어로 사전 학습된 모델 제공

#예제 5.9 spaCy 품사 태깅

- spaCy 모델 불러오기 함수(load)를 통해 모델 설정
- 객체 지향적으로 구현돼 처리한 결과를 doc객체에 저장
- doc 객체는 다시 여러 token 객체로 이뤄져 있으며, 이 token 객체에 대한 정보를 기반으로 다양한 자연어 처리 작업 수행

*spaCy에서 사용되는 태그 정리

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

5.3 하위 단어 토큰화

- 형태소 분석기의 취약점
 - 전문용어나 고유어가 많은 데이터를 처리할 때 약점을 보임
 - 모르는 단어를 적절한 단위로 나누는 것에 취약
 - 잠재적으로 어휘 사전의 크기를 크게 하고 OOV 대응에 취약
- 현대 자연어 처리에서는 신조어 발생, 오타자, 축약어 등을 고려해야함
→ 이를 해결할 방법: 하위 단어 토큰화

하위 단어 토큰화(SubwordTokenization)

하나의 단어가 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화하는 방법

- 단어의 길이를 줄일 수 있어서 처리 속도가 빨라짐
- OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제 완화
- 대표적인 방법: 바이트 페어 인코딩, 워드 피스, 유니그램 모델 등

5.3.1 바이트 페어 인코딩

텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘

- 초기에는 데이터 압축을 위해 개발
- 다이어그램 코딩(Diagram Coding)이라고도 함
- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복
 - 자주 등장하는 단어는 하나의 토큰으로 토큰화
 - 덜 등장하는 단어는 여러 토큰의 조합으로 표현
- 'abracadabra' 를 바이트 페어 인코딩 적용

- 원문: abracadabra
- Step #1: AracadAra
- Step #2: ABcadAB
- Step #3: CcadC

자주 등장하는 글자 쌍을 찾아 치환하는 대신 어휘 사전에 추가

- 말뭉치에서 바이트 페어 인코딩 적용(말뭉치에서 각 단어가 등장한 빈도 계산) [**BPE알고리즘**]

- 빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
- 어휘 사전: ['low', 'lower', 'newest', 'widest']

- 빈도 사전을 바이트 페어 인코딩으로 재구성(빈도 사전 내 모든 단어를 글자 단위로 나눔)

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 빈도 사전을 기준으로 가장 자주 등장하는 글자 쌍 찾기 → 'es'

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']

- 동일한 과정을 반복: 빈도수가 가장 높은 'es'와 't'쌍을 'est'로 병합

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']

- 이 과정을 10번 거쳤다고 가정

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

센터스피스

- 센터스피스 라이브러리: 구글에서 개발한 하위 단어 토큰라이저 라이브러리
 - 입력 데이터를 토큰화하고 단어 사전 생성
 - 워드피스, 유니코드 기반의 다양한 알고리즘을 지원
 - 하이퍼파라미터들을 제공해 세밀한 토큰나이징 기능 제공
- 코포라 라이브러리: 국립국어원이나 AI Hub에서 제공되는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리
 - 파이썬에서 사용할 수 있게 API 제공
- 라이브러리 설치

```
!pip install sentencepiece Korpora
```

토큰라이저 모델 학습

#예제 5.10 청와대 청원 데이터 다운로드

- 코포라 라이브러리는 말뭉치 불러오기(Korpora.load)를 통해 말뭉치 데이터 다운로드
- train 속성으로 학습 데이터를 가져올 수 있음
- petition에 청원 시작일, 동의 수, 제목 등의 정보가 포함 됨

#예제 5.11 학습 데이터 생성

- corpus의 get_all_texts: 본문 데이터셋을 한 번에 불러올 수 있음

#예제 5.12 토큰라이저 모델 학습

- SentencePieceTrainer로 토큰라이저 모델 학습 진행
- Train 메서드로 학습
- 문자열 입력을 통해 인자들을 전달 받음
- 센텐스피스 라이브러리의 주요한 하이퍼파라미터

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율

model_type	토크나이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

- 모델 학습 완료 후 petition_bpe.model, petition_bpe.vocab 파일 생성
- 센터스 라이브러리는 띄어쓰기나 공백도 특수문자 취급 → 토큰화 과정에서 '_'로 공백 표현

#예제 5.13 바이트 페어 인코딩 토큰화

- SentencePieceProcessor: 학습된 모델 불러오기
- tokenizer.load를 통해 petition_bpe.model 불러오기
- encode_as_pieces: 문장 토큰화
- encode_as_ids: 토큰을 정수로 인코딩해 제공

#예제 5.14 어휘 사전 불러오기

- get_piece_size: 센텐스피스 모델에서 생성된 하위 단어의 개수 반환
- id_to_piece: 정숫값을 하위 단어로 변환하는 메서드
- 출력값 확인
 - <unk> 토큰: unknown의 약자로 OOV 발생 시 매핑되는 토큰
 - <s>와 </s>는 문장의 시작 지점과 종료 지점 표시 토큰

5.3.2 워드 피스

바이트 페어 인코딩 토큰라이저와 유사 but 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합

- 새로운 하위 단어 생성 시 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택
- 정확한 하위 단어로 분리 가능
- 각 글자 상에 대한 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- f 는 빈도 함수이며 x 와 y 는 병합하려는 하위 단어
- xy 는 글자 쌍의 빈도
- 워드피스 어휘 사전 구축 방법
 - 빈도 사전 내의 모든 단어를 글자 단위로 나눈다
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']
 - 점수를 계산하여 병합
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']
 - 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 반복

토큰라이저스

- 라이브러리 설치

```
!pip install tokenizers
```

- 정규화와 사전 토큰화 제공

#예제 5.15 워드피스 토큰라이저 학습

- 정규화 방식: normalizers 모듈에 포함된 시퀀스 형식으로 인스턴스 전달
- 사전 토큰화 방식: pre_tokenizers 모듈에 포함한 클래스를 불러와 적용

*토큰라이저스 모듈의 정규화 클래스와 사전 토큰화 클래스 정리

표 5.6 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문자의 모든 대문자를 소문자로 변환
Strip()	입력 문자의 양 끝에 있는 공백 제거
StripAccents()	모든 엑센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문자 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

표 5.7 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. * 기본값: _(\u+2581)
Whitespace()	단어 경계공백과 구두점을 기준으로 사전 토큰화를 진행한다. * \w+([\~\ws])+ 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. * behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

Chapter 6. 임베딩

워드 임베딩(Word Embedding)

워드 투 벡터나 패스트 텍스트 등과 같이 단어의 의미를 학습해 표현하는 기법

동적 임베딩(Dynamic Embedding)

워드 임베딩의 단점은 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵는데, 이를 해결하기 위해 인공 신경망을 활용한 기법

6.1 언어 모델

입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델을 의미

- 언어 모델 구조

	$P(\text{만나서 반갑습니다.}) = 0.081$		$P(\text{서울특별시}) = 0.59$
Input : 안녕하세요	...	Input : 대한민국 ___ 강남구	...
	$P(\text{아니오. 괜찮습니다.}) = 0.000001$		$P(\text{아이스 아메리카노}) = 0.000001$

그림 6.1 언어 모델

6.1.1 자기회귀 언어 모델

입력된 문장들의 조건부 확률을 이용해 다음에 올 단어 예측

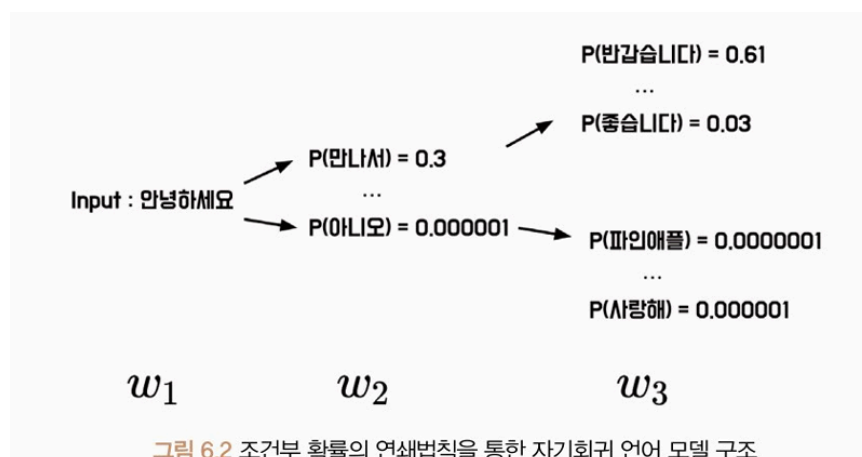
- 언어 모델에서 조건부 확률은 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것을 의미
- 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 조건부 확률의 연쇄법칙

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

- 안녕하세요 다음에 등장할 수 있는 문장을 조건부 확률의 연쇄법칙 방법으로 시각화



- 이전에 등장한 모든 토큰의 정보를 활용해 입력된 문장의 문맥정보를 파악한 후 예측

6.1.2 통계적 언어 모델

언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석

- 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측
- 일반적으로 마르코프 체인을 이용해 구현
 - 빈도 기반의 조건부 확률 모델 중 하나로 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태 예측
 - 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 대규모 자연어 데이터를 처리하는 데 효과적
- 최근 연구되는 자연어 처리 기법은 언어 모델을 활용하여 가중치를 사전학습함

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

6.2 N-gram

텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률 추정

- N개의 토큰 단위로 분석
- N이 1일때 유니그램, 2일 때 바이그램, 3일 때 트라이그램, 4이상이면 N-gram

Unigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요. 만나서. 진심으로. 반가워요

Bigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요

Trigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

- 모든 토큰 사용 X, N-1 개의 토큰만을 고려해 확률 계산

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- 시퀀스에서 연속된 n개의 단어를 추출 → 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

6.3 TF-IDF

텍스트 문서에서 특정 단어의 중요도를 계산하는 방법

- 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적 가중치를 의미
- BoW에 가중치를 부여하는 방법
- BoW: 문서나 문장을 단어의 집합으로 표현하는 방법

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- 모든 단어는 동일한 가중치를 가짐

6.3.1 단어 빈도

문서 내에서 특정 단어의 빈도수를 나타내는 값

- 단어 빈도 수식

$$TF(t, d) = count(t, d)$$

- TF 값이 높을수록 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어라 관용어로 간주할 수도 있다

6.3.2 문서 빈도

한 단어가 얼마나 많은 문서에 나타나는지를 의미

- 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수를 계산
- 문서 빈도 수식

$$DF(t, D) = count(t \in d: d \in D)$$

- DF값이 높으면 특정 단어가 많은 문서에서 등장

6.3.3 역문서 빈도

전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값

- 문서 내에서 특정 단어가 얼마나 중요한지를 나타냄
- 문서에서 특정 단어의 등장 횟수가 적으면 IDF는 상대적으로 커짐
- 역문서 빈도 수식

$$IDF(t, D) = \log\left(\frac{count(D)}{1 + DF(t, D)}\right)$$

- 1을 더해서 분모 0이 되는 것을 방지
- 로그를 취해서 너무 큰 값 방지

6.3.4 TF-IDF

앞선 문서 빈도와 역문서 빈도를 곱한 값으로 사용

- TF-IDF 수식

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

- 문서 내에서 단어가 자주 등장하지만, 전체 문서 내에서 해당 단어가 적게 등장하면 TF-IDF는 커짐
- 사이킷런 라이브러리를 활용해 계산
- TF-IDF 클래스
 - 인코딩: 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
 - 소문자 변환: 입력 받은 데이터를 소문자로 변환할지 여부
 - 불용어: 분석에 도움이 되지 않는 의미 없는 단어
 - N-gram범위: 사용할 범위
 - 최댓값 문서 빈도: 전체 문서 중 일정 횟수 이상 등장한 단어는 불용어 처리
 - 단어 사전: 미리 구축한 단어 사전이 있으면 해당 단어 사전 사용

#예제 6.2 TF-IDF 계산

- vocabulary_ : TF-IDF에 사용된 단어 사전
- 문서마다 중요한 단어만 추출 가능
- 벡터값을 활용해 문서 내 핵심 단어 추출 가능
- 그러나 문장의 순서나 문맥 고려 X